

Развертывание и настройка NoSQL базы данных MongoDB в режиме Replica Set

Курсовой проект по курсу «Администрирование Unix-подобных систем»

Выполнили:

Мизиев Рашид Саит-Алиевич
Сорокина Диана Борисовна
Деккер Полина Игоревна

Южный федеральный университет
Физический факультет

2026

Распределение ролей и индивидуальный вклад

- **Мизиев Рашид — Системный администратор/ ТехЛид**

Чем занимался в проекте: Разработка архитектуры распределенного кластера (топология PSA). Написание декларативного манифеста `docker-compose.yml`, профилирование и оптимизация параметров СУБД в `mongod.conf`, конфигурация ограничений оперативной памяти для движка `WiredTiger`.

- **Сорокина Диана — DevOps-инженер**

Чем занимался в проекте: Автоматизация процессов развертывания и инициализации окружения хоста (`bootstrap.sh`). Скриптование процессов «горячего» резервного копирования и ротации бэкапов (`backup.sh`). Настройка CI/CD пайплайнов автоматической валидации и тестирования в GitHub Actions.

- **Деккер Полина — Специалист по информационной безопасности**

Чем занимался в проекте: Реализация многоуровневой стратегии безопасности (Hardening). Создание сценария настройки межсетевого экрана хоста (`hardening.sh`), изоляция частных виртуальных сетей Docker, генерация и внедрение `keyFile` авторизации узлов репликации.

Актуальность проекта и проблематика

- **Высокая доступность:** В современных нагруженных системах падение одиночного сервера БД означает простой всей бизнес-логики.
- **NoSQL решения:** При работе с неструктурированными или часто меняющимися данными СУБД MongoDB предоставляет гибкость и горизонтальное масштабирование.
- **Методология GitOps и IaC:** Ручная настройка конфигурационных файлов и распределенных кластеров приводит к «дрейфу конфигураций» (configuration drift). Необходима 100% автоматизация.

Проблема единой точки отказа

Классическая одиночная инсталляция СУБД уязвима к аппаратным сбоям хоста. Требуется внедрение механизмов автоматического переключения ролей (Failover).

Цель работы

Проектирование, развертывание, оптимизация и защита распределенного кластера NoSQL СУБД MongoDB высокой доступности в контейнеризированной среде под управлением ОС Linux.

Задачи для достижения цели:

- 1 Разработка архитектуры и развертывание кластера MongoDB Replica Set.
- 2 Автоматизация подготовки окружения и ограничения ресурсов.
- 3 Построение системы многоуровневой безопасности (UFW, keyFile).
- 4 Разработка сценариев «горячего» резервного копирования и тестирования.
- 5 Настройка непрерывной интеграции (CI/CD) на базе GitHub Actions.

Архитектура распределенного кластера

Для исключения единой точки отказа выбрана официальная топология **Replica Set** с конфигурацией PSA (Primary-Secondary-Arbiter):

- **mongo-primary**: Главный узел, принимает все операции записи и чтения по умолчанию. Приоритет выборности: `priority: 2`.
- **mongo-secondary**: Вторичный узел, зеркалирует журнал операций (Oplog) лидера. Готов занять его место при сбое. Приоритет: `priority: 1`.
- **mongo-arbiter**: Узел-арбитр. Не хранит данные и не реплицирует логику, участвует исключительно в голосовании при выборах лидера. Флаг: `arbiterOnly: true`.

Безопасность сетевого контура реализована через виртуализацию Docker:

- Создана выделенная изолированная сеть backend-net драйвера bridge.
- Все три ноды MongoDB взаимодействуют исключительно внутри этой подсети.
- Внешний доступ к портам 27017 узлов закрыт для хост-машины и внешней сети.

Панель администрирования

Для визуального контроля в сеть интегрирован контейнер **Mongo Express**. Доступ к его веб-интерфейсу (8081) защищен базовой HTTP-аутентификацией через переменные окружения.

Автоматизация развертывания: Декларативный манифест

Инфраструктура описана как код в файле `docker-compose.yml`.
Применяются официальные стабильные образы `mongo:6.0.28`.

Фрагмент описания сервиса Primary:

```
services:
  mongo-primary:
    image: mongo:6.0.28
    container_name: mongo-primary
    restart: always
    command: ["mongod", "--bind_ip_all", "--replSet", "${MONGO_REPLICA_SET_NAME}", "--keyFile", "/opt/keyfile/replica.key"]
    volumes:
      - mongo_primary_data:/data/db
      - ../config/mongodb/mongod.conf:/etc/mongod.conf:ro
      - ../deploy/docker/mongodb/keys/replica.key:/opt/keyfile/replica.key:ro
    networks:
      - backend-net
```

Первичная подготовка среды (bootstrap.sh)

Скрипт `bootstrap.sh` полностью исключает ручную подготовку директорий серверов перед развертыванием:

- 1 Проверяет наличие локального файла секретов и настроек `.env`.
- 2 Создает иерархию каталогов для конфигураций и ключей.
- 3 Автоматически генерирует высокоэнтропийный ключ репликации через OpenSSL.

Генерация ключа взаимной авторизации узлов:

```
if [[ ! -f "${KEY_FILE}" ]]; then
  openssl rand -base64 756 > "${KEY_FILE}"
  chmod 400 "${KEY_FILE}"
fi
```


Оптимизация производительности СУБД

По умолчанию движок **WiredTiger** резервирует под внутренний кэш значительный объем ОЗУ хоста, что в контейнерной среде при запуске 3-х нод ведет к падению хоста из-за **OOM Killer**.

В конфигурационном файле `mongod.conf` произведено жесткое ограничение ресурсов.

Конфигурация лимитов кэша памяти:

```
storage:  
  dbPath: /data/db  
  engine: wiredTiger  
  wiredTiger:  
    engineConfig:  
      cacheSizeGB: 1.0
```

Обеспечение безопасности (Hardening)

Защита системы реализована на уровне ОС хоста и на уровне самого приложения:

- **Права доступа:** На сгенерированный `replica.key` накладываются строгие права `chmod 400` (чтение только владельцем), без которых движок MongoDB отклоняет запуск.
- **Авторизация СУБД:** В блоке `security` активирована директива `authorization: enabled`.
- **Хостовый Межсетевой экран (UFW):** Сценарий `hardening.sh` настраивает брандмауэр по принципу «запрещено все, кроме явно разрешенного» — открыты порты 22 (SSH), 80, 443. Порт базы данных закрыт снаружи.

Автоматизация резервного копирования

Разработан сценарий `backup.sh`, реализующий стратегию «горячего» бэкапа (Hot Backup) без остановки контейнеров кластера.

- Выполняет утилиту `mongodump` внутри контейнера `mongo-primary` с флагом `-gzip` для сжатия.
- Безопасно извлекает архив на хост-машину через `docker cp`.
- **Ротация архивов:** Автоматически очищает старые резервные копии, предотвращая переполнение дискового пространства хоста.

Автоматическая ротация (удаление копий старше 7 дней):

```
find "${BACKUP_DIR}" -type f -name "*.gz" -mtime +7 -exec rm {} \;
```

Интеграционное тестирование (test_mongo.sh)

Для валидации корректности сборки репликации и проверки доступности СУБД перед продакшн-эксплуатацией разработан автоматический тест. Скрипт проверяет систему в три этапа:

- **Этап 1:** Циклическая проверка готовности сетевого сокета СУБД (`db.adminCommand('ping')`) с таймаутом ожидания в 60 секунд.
- **Этап 2:** Проверка статуса инициализации Replica Set.
- **Этап 3:** Тест сквозной операции записи (`db.test.insertOne()`) для проверки доступности Primary-ноды на чтение и запись.

Запрос статуса репликации в скрипте:

```
RS_STATUS=$(docker exec mongo-primary mongosh --quiet --eval "rs.status().ok")
```

Конвейер CI/CD: Статический анализ кода

Пайплайн настроен в среде **GitHub Actions** (конфигурация `pr-validation.yml`) и запускается автоматически на каждый Pull Request. На первом этапе обрабатывают линтеры качества кода:

- **yamllint**: Проверяет структуру, синтаксические отступы и стандарты оформления во всех YAML-файлах проекта и конфигурациях Docker Compose.
- **ShellCheck**: Анализирует bash-скрипты автоматизации на наличие скрытых логических ошибок, переполнений переменных и соответствие стандарту POSIX.
- **Hadolint**: Проверяет инструкции Dockerfile на оптимизацию слоев сборки образов.

Эфемерное тестирование (In-Pipeline Tests)

После успешного прохождения линтеров GitHub Actions разворачивает виртуальную машину-раннер на Ubuntu, собирает «с нуля» весь стек контейнеров через Docker Compose и прогоняет разработанный скрипт `test_mongo.sh`. При падении хотя бы одного теста слияние кода в ветку `main` блокируется.

- **Управление релизами (release.yml):** При публикации нового тега версии (`v*`) пайплайн автоматически сверяет временные метки изменений исходных кодов и скомпилированных документов. Это гарантирует, что в релиз уходят только актуальные и проверенные файлы.

Результаты выполнения курсового проекта

- Спроектирована и развернута отказоустойчивая NoSQL-инфраструктура на базе распределенного кластера MongoDB Replica Set.
- Обеспечена сетевая изоляция и защита данных (UFW, права 400, аутентификация по ключам), что снижает риски несанкционированного доступа.
- Вся инфраструктура управляется декларативно в рамках концепции IaC через Docker Compose, Makefile и автоматизированные bash-сценарии.
- Внедрен полноценный цикл автоматических проверок (линтеры, интеграционные тесты) в облачном пайплайне GitHub Actions.

Спасибо за внимание! Вопросы.